

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет программной инженерии и компьютерной техники Направление подготовки

09.03.04 Программная инженерия

Дисциплина «Бизнес-логика программных систем»

ОТЧЕТ

По лабораторной работе 2

Вариант 111

Преподаватель:

Кривоносов Егор Дмитриевич

Выполнил:

Алхимовици Арсений, Юсупова Алиса

Группа:

Р3310, 3313

Санкт-Петербург

2026 г.

Текст задания

Доработать приложение из лабораторной работы 1, реализовав в нём управление транзакциями и разграничение доступа к операциям бизнес-логики в соответствии с заданной политикой доступа.

Управление транзакциями:

1. Переработать согласованные с преподавателем прецеденты (или по согласованию с ним разработать новые), объединив взаимозависимые операции в рамках транзакций.
2. Управление транзакциями реализовать с помощью Spring JTA.
3. В реализованных (или модифицированных) прецедентах использовать декларативное управление транзакциями.
4. В качестве менеджера транзакций использовать Narayana.

Разграничение доступа:

1. Разработать, специфицировать и согласовать с преподавателем набор привилегий для разграничения доступа к операциям.
2. Специфицировать и согласовать с преподавателем набор ролей для доступа к операциям бизнес-логики.
3. Реализовать модель разграничений на базе Spring Security + JAAS. Учётные записи хранить в XML; для доступа к API использовать JWT.

Правила выполнения работы:

Все изменения, внесённые в реализуемый бизнес-процесс, должны быть учтены в описывающей его модели, REST API и наборе скриптов для тестирования публичных интерфейсов модуля. Доработанное приложение необходимо развернуть на сервере helios.

Исходный код

<https://github.com/senya-2011/mts-online-shop>

Модель потока управления для автоматизируемого бизнес-процесса

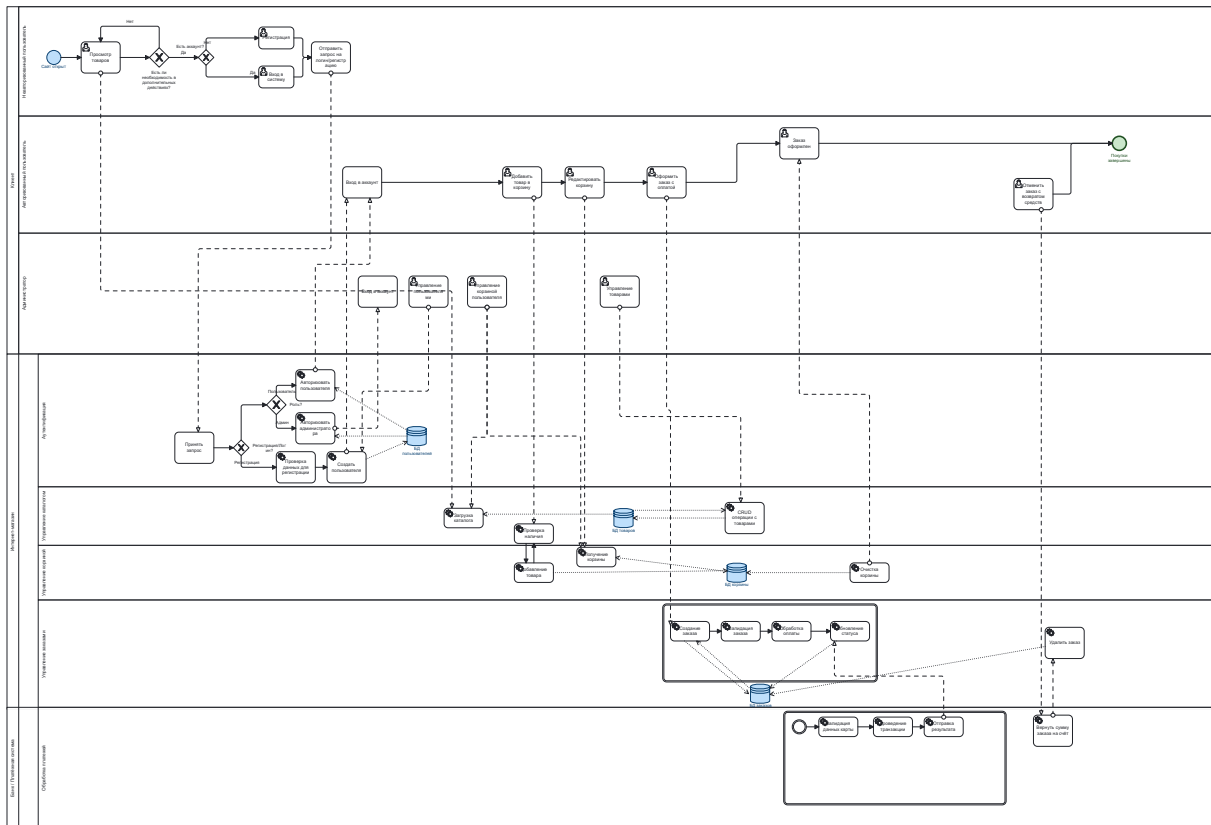


Рисунок 1: BPMN диаграмма

UML-диаграммы

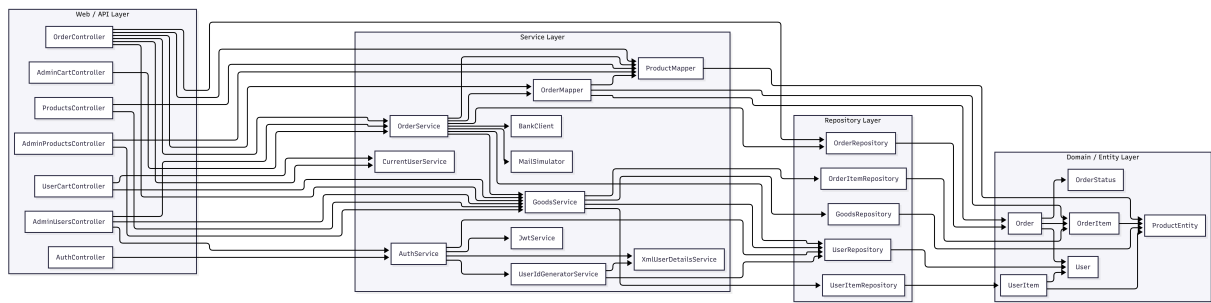


Рисунок 2: UML-диаграмма классов

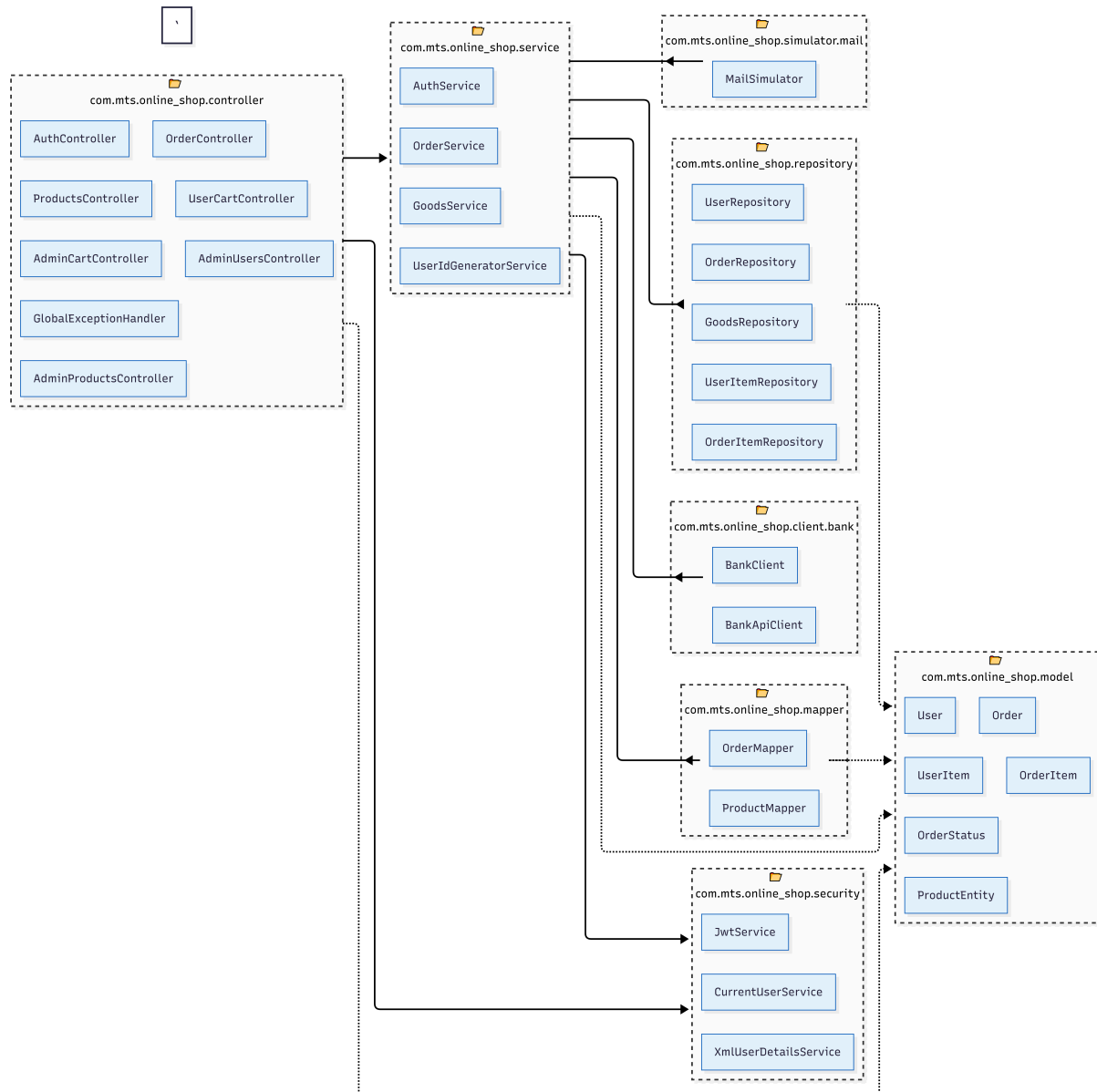


Рисунок 3: UML-диаграмма пакетов

Спецификация REST API

Спецификация REST API задана в формате OpenAPI 3.0.3 (файл `api/openapi.yml`).

Auth (без JWT)

Метод	Путь	Описание
POST	/auth/login	Вход. Тело: LoginRequest (login, password). Ответ: 200 LoginResponse (accessToken).
POST	/auth/register	Регистрация. Тело: RegisterRequest (login, email, password, name). Ответ: 201 MessageResponse.

Products (без JWT)

Метод	Путь	Описание
GET	/products	Каталог товаров. Ответ: 200 ProductListResponse: items, total, page, size.
GET	/products/{id}	Товар по идентификатору. Ответ: 200 Product (id, name, price).

Cart (Bearer JWT)

Метод	Путь	Описание
GET	/cart	Корзина текущего пользователя. Ответ: 200 CartResponse (items с itemId и product, total, page, size).
POST	/cart/items	Добавить товар в корзину. Тело: AddCartItemRequest (productId). Ответ: 201 MessageResponse.
DELETE	/cart/items/{itemId}	Удалить позицию корзины. Ответ: 200 MessageResponse.
DELETE	/cart/clear	Очистить корзину. Ответ: 200 MessageResponse.

Orders (Bearer JWT)

Метод	Путь	Описание
GET	/orders	Список заказов пользователя. Ответ: 200 OrderListResponse (items — OrderResponse, total, page, size).
POST	/orders/create	Создание заказа из корзины с оплатой в одной транзакции. Тело: PaymentRequest (cardNumber, expiresAt, cvv). Ответ: 201 OrderResponse; 400/500.
POST	/orders/{orderId}/cancel	Отмена заказа с возвратом средств. Ответ: 200 MessageResponse.

Admin Users (Bearer JWT; в спецификации ответ 403 при отсутствии прав администратора)

Метод	Путь	Описание
GET	/admin/users	Список пользователей. Query: page, size. Ответ: 200 UserListResponse; 401/403.
GET	/admin/users/{userId}	Пользователь по id. Ответ: 200 UserResponse; 401/403/404.
GET	/admin/users/{userId}/cart	Корзина пользователя. Ответ: 200 CartResponse; 401/403/404.
DELETE	/admin/users/{userId}/cart/clear	Очистить корзину пользователя. Ответ: 200 MessageResponse; 401/403/404.

Admin Orders (Bearer JWT; 403 при отсутствии прав администратора)

Метод	Путь	Описание
GET	/admin/orders	Все заказы. Query: page, size. Ответ: 200 OrderListResponse; 401/403.
POST	/admin/orders/{orderId}/cancel	Отмена заказа с возвратом. Ответ: 200 MessageResponse; 401/403/404.

Admin Products (Bearer JWT; 403 при отсутствии прав администратора)

Метод	Путь	Описание
GET	/admin/products	Список товаров (в т.ч. скрытые). Query: page, size. Ответ: 200 ProductListResponse; 401/403.
POST	/admin/products	Создать товар. Тело: CreateProductRequest (name, price). Ответ: 201 Product; 400/401/403.
PUT	/admin/products/{productId}	Обновить товар. Тело: UpdateProductRequest (name, price). Ответ: 200 Product; 400/401/403/404.
DELETE	/admin/products/{productId}	Удалить товар. Ответ: 200 MessageResponse; 401/403/404.

Выводы по работе

В соответствии с текстом задания к лабораторной работе 2 доработано приложение из ЛР1: добавлены **управление транзакциями и разграничение доступа**.

Транзакции (Spring JTA, Narayana): взаимозависимые операции оформлены в прецедентах с использованием **декларативного** управления (@Transactional) под менеджером транзакций Narayana; для ключевых сценариев (создание и оплата заказа, отмена с возвратом и др.) обеспечена согласованность данных при обращении к БД и внешнему банковскому сервису.

Доступ (Spring Security + JAAS): согласованы и заданы **роли** (в т.ч. USER и ADMIN) и **привилегии** для операций; учётные данные и роли загружаются из **XML**; для REST используется **JWT** (Bearer). Пользовательские сценарии (каталог, корзина, заказы) и административные (/admin/...) разделены на уровне API и политики безопасности.

Артефакты: обновлены модель процесса (BPMN), спецификация **OpenAPI 3.0.3** (api/openapi.yml) с описанием публичных и админских эндпоинтов, исходный код и развёртывание на **helios** (GitHub Actions). Реализация REST: контроллеры ProductsController, UserCartController, OrderController, админские AdminUsersController, AdminCartController, AdminProductsController маппинг сущностей — MapStruct; уведомления — почтовый симулятор при необходимости. Настроены тесты (Kotest) и CI.